

RAPPORT DE PROJET : MIGRATION D'ECOBUDGET VERS KOTLIN MULTIPLATFORM (KMP)

1. Présentation du projet

EcoBudget est une application Android de gestion budgétaire développée en Kotlin. Dans le cadre du projet, l'application a été migrée progressivement vers **Kotlin Multiplatform (KMP)** afin de rendre une partie du code réutilisable sur plusieurs plateformes, notamment Android et iOS.

L'objectif principal de cette migration est de déplacer les éléments communs de l'application vers le répertoire :

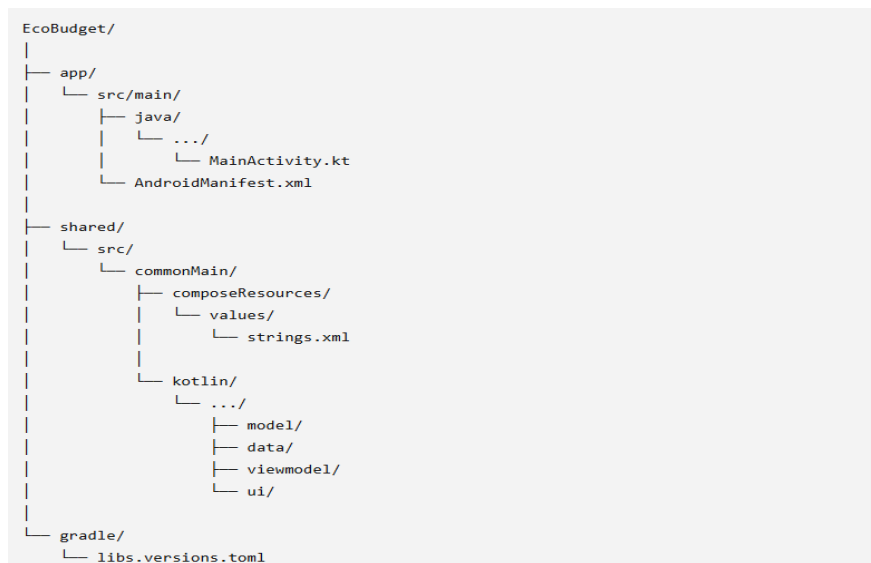
```
shared/src/commonMain/
```

Cependant, le code initial utilisait certaines classes et bibliothèques propres à Android ou à la JVM. Ces dépendances empêchaient leur compilation dans `commonMain`.

La migration a donc consisté à identifier ces dépendances et à les remplacer par des solutions compatibles avec l'approche multiplateforme.

2. Architecture après migration

Le projet est organisé de la manière suivante :



Le module `:app` contient les éléments spécifiques à Android, tandis que `:shared` contient le code pouvant être partagé entre les plateformes.

3. Migration des fichiers vers `commonMain`

3.1 Fichier `YearMonth.kt`

Problème rencontré

Le fichier `YearMonth.kt` utilisait des classes de l'environnement Java pour manipuler les dates.

Certains imports de type :

```
import java.util.Calendar
import java.text.SimpleDateFormat
```

ne sont pas disponibles dans `commonMain`.

Le code pouvait donc fonctionner sur Android, mais sa compilation dans le code commun échouait.

Choix technique appliqué

La gestion des dates a été adaptée avec la bibliothèque :

```
kotlinx-datetime
```

Les types multiplateformes tels que :

- `LocalDate`
- `LocalDateTime`
- `Instant`
- `TimeZone`

peuvent ainsi être utilisés à la place des API Java.

Justification

`kotlinx-datetime` est conçue pour Kotlin Multiplatform et permet d'utiliser une logique commune de gestion des dates sur différentes plateformes.

La solution reste donc compatible avec Android tout en permettant au même code d'être utilisé sur iOS.

3.2 Fichier `Transaction.kt`

Problème rencontré

Le modèle `Transaction` utilisait certaines fonctionnalités dépendantes de la JVM, notamment pour la génération ou la manipulation des identifiants.

L'utilisation directe de :

```
java.util.UUID
```

créait une dépendance à l'environnement Java.

Choix technique appliqué

La génération des identifiants a été adaptée afin d'utiliser des mécanismes compatibles avec Kotlin Multiplatform, notamment les fonctionnalités de :

```
kotlin.random.Random
```

et les API temporelles multiplateformes lorsque cela est nécessaire.

Justification

La solution ne dépend plus directement de `java.util.UUID`.

Le modèle `Transaction` peut donc être placé dans `commonMain` et être utilisé aussi bien par Android que par une future implémentation iOS.

3.3 Fichier `Category.kt`

Problème rencontré

Le modèle `Category` faisait partie du code initial Android et devait être déplacé vers le module partagé.

Le principal problème concernait les dépendances indirectes à des classes ou ressources spécifiques à Android.

Choix technique appliqué

Le modèle a été simplifié afin de ne conserver dans `commonMain` que des types Kotlin génériques et multiplateformes.

Les dépendances propres à Android ont été supprimées.

Justification

Un modèle métier ne doit pas dépendre de la plateforme.

Cette modification permet donc à `Category` d'être utilisé par les différentes interfaces de l'application.

3.4 Fichier `EcoBudgetViewModel.kt`

Problème rencontré

Le `ViewModel` initial était principalement conçu pour Android et pouvait utiliser des composants provenant directement de l'écosystème Android.

Le déplacement direct du fichier dans `commonMain` provoquait donc des problèmes de compilation.

Choix technique appliqué

Le `ViewModel` a été migré vers le module partagé en utilisant les API compatibles avec Kotlin Multiplatform, notamment :

```
StateFlow  
combine  
viewModelScope
```

Le `EcoBudgetViewModel` est ainsi placé dans :

```
shared/src/commonMain/
```

Justification

Le `ViewModel` peut ainsi centraliser la logique de présentation et l'état de l'application sans être directement lié à une implémentation Android.

Cela permet de réutiliser la même logique sur Android et iOS.

3.5 Fichiers de l'interface utilisateur

Les différents composables de l'application ont également été déplacés vers `commonMain`.

Problème rencontré

L'interface initiale pouvait contenir des imports spécifiques à Jetpack Compose Android ou à certaines API Android.

Ces imports n'étaient pas nécessairement disponibles dans un environnement multiplateforme.

Choix technique appliqué

Les composants ont été adaptés à **Compose Multiplatform**.

Un composable principal :

```
EcoBudgetApp()
```

a été utilisé comme point d'entrée de l'interface partagée.

Justification

Compose Multiplatform permet de partager une grande partie de l'interface entre les plateformes.

L'interface peut donc être développée dans `commonMain` tout en restant utilisable sur Android et potentiellement sur iOS.

3.6 Fichier string.xml

Problème rencontré

Les ressources étaient initialement organisées selon le système de ressources Android.

Elles n'étaient donc pas directement accessibles depuis le code commun `commonMain`.

Choix technique appliqué

Le fichier a été déplacé vers :

```
shared/src/commonMain/composeResources/values/strings.xml
```

Les chaînes sont ensuite accessibles avec :

```
stringResource(Res.string.nom_de_la_chaine)
```

Justification

`composeResources` permet de gérer des ressources utilisables par Compose Multiplatform.

Les chaînes peuvent donc être utilisées dans le code partagé au lieu d'être limitées au module Android.

Vérification de la compatibilité

Après chaque migration, le projet a été recompilé afin d'identifier les éventuelles dépendances encore liées à Android ou à la JVM.

La démarche suivie était donc :

Fichier Android

↓

Identification des imports incompatibles
↓
Remplacement par une solution multiplateforme
↓
Déplacement vers commonMain
↓
Compilation
↓
Correction des éventuelles erreurs
↓
Validation

Cette méthode permet de migrer progressivement le projet plutôt que de déplacer tous les fichiers simultanément.

Conclusion

La migration d'EcoBudget vers Kotlin Multiplatform a principalement consisté à **identifier et supprimer les dépendances spécifiques à Android ou à la JVM** présentes dans les fichiers destinés à `commonMain`.

Les principales adaptations concernent la gestion des dates, les identifiants, les ViewModels, l'état de l'interface, les composants Compose et les ressources.

L'utilisation de bibliothèques et d'API compatibles avec Kotlin Multiplatform permet de conserver le fonctionnement actuel sur Android tout en préparant le projet à une future utilisation sur **iOS**.

L'architecture obtenue sépare ainsi les éléments spécifiques à la plateforme des éléments communs de l'application, ce qui facilite la maintenance et l'évolution future d'EcoBudget.